

Doc. No.:	P0063R1	
Date:	2016-02-12	
Reply to:	Clark Nelson	Hans-J. Boehm
Email:	clark.nelson@intel.com	hboehm@google.com
Audience:	Library Evolution	

C++17 should refer to C11 instead of C99

Introduction

This document is a follow-on to [P0063R0](#), from the pre-Kona mailing. It goes into the details of the “easy part” described therein. Much of its rationale is not repeated here.

Signal handler restrictions

18.10p10 effectively says that a signal handler must be written in the common subset of C and C++. But C++ and C11 both have thread-local storage. Referencing C11 instead of C99 might be taken to suggest that it would be OK for a signal handler to reference a thread-local object.

It turns out that there is a technicality/loophole by which this can be dodged. Because the associated keyword is spelled differently in C and C++, technically thread-local storage is not actually in the common subset of the two languages – even though both languages support the feature.

While this state would not provide a satisfying long-term resolution, it is considered adequate to separate the deep technical issues of signal handlers from the broader, more editorial issues of updating the normative reference. Hans is writing P0270 to address signal handlers.

This document also doesn't go into interoperability of threads or atomics in a mixed C/C++ program.

Header coverage

C11 made the `<complex.h>` header optional. Currently in C++, the `<ccomplex>` header maps directly to the `<complex>` header. Simply deleting the requirement for a `<ccomplex>` header is simpler, and may be better, than making it optional. Of course an implementation would not need to change in order to conform to the new state of affairs, but programs that use `<ccomplex>` instead of `<complex>` would cease to be strictly conforming.

C11 also added five new headers:

`<stdatomic.h>`

`<threads.h>`

These are also optional in C11, and since their functionality is already provided in C++ by other means, it's not clear that inventing optional wrappers for them would be worth the effort.

`<stdalign.h>`

`<uchar.h>`

C++ already covers these two, having inherited one of them from the C Unicode TR (later incorporated into C11), and having effectively jumped the gun on the other one.

`<stdnoreturn.h>`

All this provides is a single prettifying macro:

```
#define noreturn _Noreturn
```

In C++, `[[noreturn]]` is probably already considered pretty enough.

So it's not clear that C++ needs to add anything for any of these new headers.

Annex K Bounds checking interfaces

C11 optionally added several dozen new functions to the standard library, as safer alternatives to existing standard C library functions. For example, `strcat_s` was added as an alternative to `strcat`.

The declarations of these functions were added to the existing headers; backward compatibility was provided by requiring that

they be controlled by the setting of a macro: `__STDC_WANT_LIB_EXT1__`. A program can suppress them by defining the macro to zero, or request them by defining the macro to one. Whether they are declared if the macro is not defined by the program is implementation-defined.

At the most recent WG14 meeting, a proposal was considered ([N1967](#)) that these functions should be removed from a future revision of the standard.

The wording changes in this paper don't address them.

Reference mechanism

Consider 17.5.1.5 paragraph 1:

Paragraphs labeled “See also:” contain cross-references to the relevant portions of this International Standard and the ISO C standard, which is incorporated into this International Standard by reference.

Elsewhere it says “as if by inclusion”; why does it here say “by reference”?

Annex C Compatibility

The wording changes in this paper don't yet address Annex C.5.

Working draft changes

1.2 Normative references

Change paragraph 1:

- ...
- ...
- ISO/IEC 9899:19992011, *Programming languages — C*
- ISO/IEC 9899:19992011/Cor 1:20012012(E), *Programming languages — C, Technical Corrigendum 1*
- ISO/IEC 9899:1999/Cor.2:2004(E), *Programming languages — C, Technical Corrigendum 2*
- ISO/IEC 9899:1999/Cor.3:2007(E), *Programming languages — C, Technical Corrigendum 3*
- ...
- ISO/IEC TR 19769:2004, *Information technology — Programming languages, their environments and system software interfaces — Extensions for the programming language C to support new character data types*

Change paragraph 2:

The library described in Clause 7 of ISO/IEC 9899:19992011 and Clause 7 of ISO/IEC 9899:1999/Cor.1:2001 and Clause 7 of ISO/IEC 9899:1999/Cor.2:2003 is hereinafter called the *C standard library*.¹

Delete paragraph 3:

The library described in ISO/IEC TR 19769:2004 is hereinafter called the *C Unicode TR*.

17.6.1.2 Headers

Change paragraph 3:

The facilities of the C standard Library are provided in 26 25 additional headers, as shown in Table 15.

Change table 15:

<cassert>	<ctype>	<csignal>	<stdio>	<wchar>
<complex>	<iso646>	<stdalign>	<stdlib>	<wctype>
<cctype>	<climits>	<stdarg>	<string>	
<errno>	<locale>	<stdbool>	<tgmath>	
<cfenv>	<cmath>	<stddef>	<ctime>	
<float>	<setjmp>	<stdint>	<uchar>	

Change paragraph 4:

Except as noted in Clauses 18 through 30 and Annex D, the contents of each header *cname* shall be the same as that of the corresponding header *name.h*, as specified in the C standard library (1.2) or the C Unicode TR, as appropriate, as if

by inclusion. In the C++ standard library, however, the declarations (except for names which are defined as macros in C) are within namespace scope (3.3.6) of the namespace `std`. It is unspecified whether these names (including any overloads added in Clauses 18 through 30 and Annex D) are first declared within the global namespace scope and are then injected into namespace `std` by explicit *using-declarations* (7.3.3).

18.2 Types

Change reference to C standard for paragraph 9:

See also: ... ISO C ~~7:1.6~~ 7.19.

18.3.3 C library

Change table 32:

DBL_DIG	DBL_MIN_EXP	FLT_MAX_EXP	LDBL_MANT_DIG
DBL_EPSILON	DECIMAL_DIG	FLT_MIN	LDBL_MAX_10_EXP
DBL_MANT_DIG	FLT_DIG	FLT_MIN_10_EXP	LDBL_MAX_EXP
DBL_MAX	FLT_EPSILON	FLT_MIN_EXP	LDBL_MAX
DBL_MAX_10_EXP	FLT_EVAL_METHOD	FLT_RADIX	LDBL_MIN
DBL_MAX_EXP	FLT_MANT_DIG	FLT_ROUNDS	LDBL_MIN_10_EXP
DBL_MIN	FLT_MAX	LDBL_DIG	LDBL_MIN_EXP
DBL_MIN_10_EXP	FLT_MAX_10_EXP	LDBL_EPSILON	
<u>DBL_HAS_SUBNORM</u>	<u>FLT_HAS_SUBNORM</u>	<u>LDBL_HAS_SUBNORM</u>	
<u>DBL_DECIMAL_DIG</u>	<u>FLT_DECIMAL_DIG</u>	<u>LDBL_DECIMAL_DIG</u>	
<u>DBL_TRUE_MIN</u>	<u>FLT_TRUE_MIN</u>	<u>LDBL_TRUE_MIN</u>	

It looks like all the functionality represented by these macros is already available from `numeric_limits`.

Change references to C standard:

See also: ISO C ~~7:1.5~~, 5.2.4.2.2, 5.2.4.2.1.

18.5 Start and termination

Change paragraph 12:

... [*Note*: The standard file buffers are not flushed. See: ISO C ~~7:20.4.4~~ 7.22.4.5. —*end note*]

Change reference to C standard for paragraph 4:

See also: ... ISO C ~~7:10.4~~ 7.22.4.

18.6.1.4 Data races

Change paragraph 1:

For purposes of determining the existence of data races, the library versions of `operator new`, user replacement versions of global `operator new`, the C standard library functions `aligned_alloc`, `calloc` and `malloc`, the library versions of `operator delete`, user replacement versions of `operator delete`, the C standard library function `free`, and the C standard library function `realloc` shall not introduce a data race (17.6.5.9). Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before (1.10) the next allocation (if any) in this order.

18.6.1.1 also references `malloc`, and reference to `aligned_alloc` should be made as well; that issue is addressed in P0035 (Dynamic memory allocation for over-aligned data).

18.10 Other runtime support

Change reference to C standard for paragraph 3:

See also: ISO C ~~4:8.1.1~~ 7.16.1.1.

Change references to C standard for paragraph 4:

See also: ISO C ~~7:10.4~~, 7.8, 7.6, 7.13 ~~7.12~~.

7.10.4 referred to the `exit` family of functions; 7.8 to `<stdarg.h>`; and 7.12 (possibly) to `<time.h>`. These other references may be relevant elsewhere, but not to this paragraph, which is solely about `setjmp` and `longjmp`.

19.3 Assertions

Change paragraph 3:

The contents are the same as the Standard C library header `<assert.h>`, except that a macro named `static_assert` is not defined in header `<cassert>` or in header `<assert.h>`.

Note: This kind of problem is handled in a slightly different way in 18.10p7-8.

20.7.13 C library

Change table 45:

Functions: `calloc` `malloc` `aligned_alloc`
`free` `realloc`

Change paragraph 3:

The functions `aligned_alloc()`, `calloc()`, `malloc()`, and `realloc()` do not attempt to allocate storage by calling `::operator new()` (18.6).

Change the reference to the C standard for paragraph 4:

See also: ISO C Clause ~~7.11.2~~ 7.22.3.

Change paragraph 5:

Storage allocated directly with `aligned_alloc()`, `malloc()`, `calloc()`, or `realloc()` is implicitly declared reachable (see 3.7.4.3) on allocation,

Change the reference to the C standard for paragraph 7:

See also: ISO C Clause ~~7.11.2~~ 7.24.

20.12.8 Date and time functions

Change table 60:

Macros: `NULL` `CLOCKS_PER_SEC` `TIME_UTC`
Types: `size_t` `clock_t` `time_t`
Struct: `tm` `timespec`
Functions:
`asctime` `clock` `difftime` `localtime` `strftime`
`ctime` `gmtime` `mktime` `time` `timespec_get`

Change the references to the C standard for paragraph 2:

See also: ISO C Clause ~~7.12~~ 7.27, ~~Amendment 1 Clause 4.6.4~~.

21.8 Null-terminated sequence utilities

Change the references to the C standard for paragraph 15:

See also: ISO C ~~7.3, 7.10.7, 7.10.8, and 7.11~~. ~~Amendment 1 4.4, 4.5, and 4.6~~ 7.4, 7.22.7, 7.22.8, 7.24, 7.28, 7.29 and 7.30.

22.6 C library locales

Change the reference to the C standard for paragraph 3:

See also: ISO C Clause ~~7.4~~ 7.11.

25.5 C library algorithms

Change the reference to the C standard for paragraph 4:

See also: ISO C Clause ~~7.10.5~~ 7.22.5.

26.3 The floating-point environment

For editorial consistency, replace everything from section header 26.3.1 through paragraph 2 with the following:

Table N describes header `<cfenv>`.

Table N — Header `<cfenv>` synopsis

Macros:

FE_DIVBYZERO	FE_OVERFLOW	FE_DOWNWARD	FE_UPWARD
FE_INEXACT	FE_UNDERFLOW	FE_TONEAREST	FE_DFL_ENV
FE_INVALID	FE_ALL_EXCEPT	FE_TOWARDZERO	

Types: `fenv_t` `fexcept_t`

Functions:

<code>feclearexcept</code>	<code>fesetexceptflag</code>	<code>fesetround</code>	<code>fesetenv</code>
<code>fegetexceptflag</code>	<code>fetestexcept</code>	<code>fegetenv</code>	<code>feupdateenv</code>
<code>feraiseexcept</code>	<code>fegetround</code>	<code>feholdexcept</code>	

The contents of this header are the same as the Standard C library header `<fenv.h>`.

26.8 **C library**

Change paragraph 1:

The header `<ctgmath>` simply includes the headers ~~`<complex>`~~ `<complex>` and `<cmath>`.

Change paragraph 2:

[*Note:* The overloads provided in C by type-generic macros are already provided in ~~`<complex>`~~ `<complex>` and `<cmath>` by “sufficient” additional overloads. —*end note*]

Change the references to the C standard for paragraph 11:

See also: ISO C 7.5, ~~7.10.2, 7.10.6~~ 7.12, 7.22.2, 7.22.6.

27.4.1 **Overview**

Change paragraph 3:

Mixing operations on corresponding wide- and narrow-character streams follows the same semantics as mixing such operations on FILEs, as specified in ~~Amendment 1~~ of the ISO C standard.

27.9.2 **C library files**

Change paragraph 1:

Table 133 describes header `<stdio>`. ~~[*Note:* C++ does not define the function `gets`. —*end note*]~~

Add a new paragraph following paragraph 1:

The contents of this header are the same as the Standard C library header `<stdio.h>`.

Change table 133:

<code>clearerr</code>	<code>fopen</code>	<code>fsetpos</code>	<code>putchar</code>	<code>snprintf</code>	<code>vscanf</code>
<code>fclose</code>	<code>fprintf</code>	<code>ftell</code>	<code>puts</code>	<code>sprintf</code>	<code>vsnprintf</code>
<code>feof</code>	<code>fputc</code>	<code>fwrite</code>	<code>remove</code>	<code>sscanf</code>	<code>vsprintf</code>
<code>ferror</code>	<code>fputs</code>	<code>getc</code>	<code>rename</code>	<code>tmpfile</code>	<code>vsscanf</code>
<code>fflush</code>	<code>fread</code>	<code>getchar</code>	<code>rewind</code>	<code>tmpnam</code>	<u><code>vfscanf</code></u>
<code>fgetc</code>	<code>freopen</code>	<code>pererror</code>	<code>scanf</code>	<code>ungetc</code>	
<code>fgetpos</code>	<code>fscanf</code>	<code>printf</code>	<code>setbuf</code>	<code>vfprintf</code>	
<code>fgets</code>	<code>fseek</code>	<code>putc</code>	<code>setvbuf</code>	<code>vprintf</code>	

Change the references to the C standard for paragraph 2:

See also: ISO C 7.9, ~~Amendment 1~~ 4.6.2 7.21.

D.3 **C standard library headers**

Change paragraph 1:

For compatibility with the C standard library and the C Unicode TR, the C++ standard library provides the ~~26~~ 25 C

headers, as shown in Table 154.

Change table 154:

<assert.h>	<inttypes.h>	<signal.h>	<stdio.h>	<wchar.h>
<complex.h>	<iso646.h>	<stdalign.h>	<stdlib.h>	<wctype.h>
<ctype.h>	<limits.h>	<stdarg.h>	<string.h>	
<errno.h>	<locale.h>	<stdbool.h>	<tgmath.h>	
<fenv.h>	<math.h>	<stddef.h>	<time.h>	
<float.h>	<setjmp.h>	<stdint.h>	<uchar.h>	